Retail Shop SQL Project

Project Title: Retail Shop Database Management using SQL
Technology Used: MySQL
Database Name: retail_shop

Project Objectives

- To manage customer, product, order, and payment data efficiently using SQL.
- To perform data retrieval and filtering operations using SELECT queries.
- To analyze sales, revenue, and customer activity using aggregate functions.
- To implement joins for combining data from multiple related tables.
- To use subqueries for advanced business insights and reporting.
- To identify active and inactive customers for marketing strategies.
- To track inventory and product performance using SQL queries.
- To generate useful reports for business decision-making.

SQL Queries Used in the Project

-- BASIC
-- 1. RETRIEVE CUSTOMER NAMES AND EMAILS FOR EMAIL MARKETING

```
Input: select name, email FROM customers;
```

Output:

	name	email
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com
	Leslie Wilson	user13@example.com
	Deborah Arias	user14@example.com
	Austin Flores	user15@example.com
	Amy Landry	user16@example.com
	Randy Mooney	user17@example.com
	Jeffrey Bray	user18@example.com
	Amanda Bright	user19@example.com
	Megan Lee	user20@example.com

customers 2 ×

-- 2. VIEW COMPLETE PRODUCT CATALOG WITH ALL AVAILABLE DETAILS

```
Input: select * from products;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Pl Plant No gial	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27

products 3 ×

-- 3. List all unique product categories

```
Input: select distinct category from products;
```

Output:

	category
▶	Home
	Clothing
	Electronics
	Toys
	Books

-- 4. Show all products priced above ₹1,000

```
Input: select * from products where price > 1000;
```

Output:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	F	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43

Action Ask

products 5

-- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
Input: select * from products where price between 2000 and 5000;
```

Output:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
[illegible]	Fast Foot	Toys	2414.93	128	2025-05-11 16:02:29

products 6

-- 6. Fetch data for specific customer IDs

```
Input: select * from customers where customer_id in (2,4,6,8);
```

Output:

customer_id	name	email	phone	created_at
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
NULL	NULL	NULL	NULL	NULL

-- 7. Identify customers whose names start with the letter ‘A’

```
Input: select * from customers where name like 'A%';
```

Output:

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

-- 8. List electronics products priced under ₹3,000

```
Input: select * from products where category = 'Electronics'and price < 3000;
```

Output:

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Actually Term	2.46	40	2024-02-28 17:57:38
NULL	NULL	NULL	NULL	NULL	NULL

-- 9. Display product names and prices in descending order of price

```
Input: select name, price from products order by price desc;
```

Output:

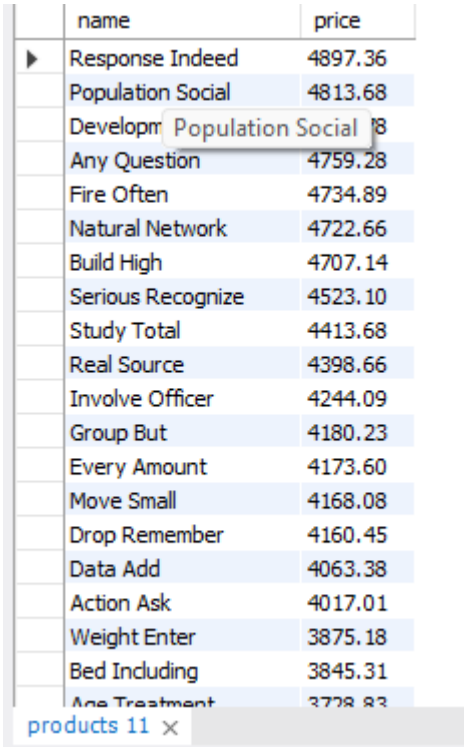
name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18
Bed Including	3845.31
Age Treatment	3728.83

products 10 ×

-- 10. Display product names and prices sorted by price and then by name

```
Input: select name, price from products order by price desc , name asc;
```

Output:



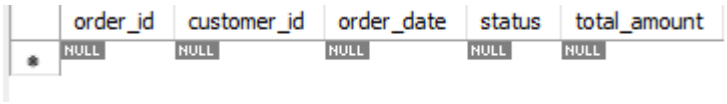
name	price
Response Indeed	4897.36
Population Social	4813.68
Developm	8
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18
Bed Including	3845.31
Age Treatment	3728.83

-- Level 2: Filtering and Formatting

--1. Retrieve orders where customer information is missing

```
Input: select * from orders where customer_id is null;
```

Output:



order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

--2. Display customer names and emails using column aliases

```
Input: select name AS FULL_NAME, email as CUSTOMER_EMAIL_ADDRESS from customers;
```

Output:

	FULL_NAME	CUSTOMER_EMAIL_ADDRESS
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com
	Leslie Wilson	user13@example.com
	Deborah Arias	user14@example.com
	Austin Flores	user15@example.com
	Amy Landry	user16@example.com
	Randy Mooney	user17@example.com
	Jeffrey Bray	user18@example.com
	Amanda Bright	user19@example.com
	Megan Lee	user20@example.com

customers 13 ×

--3. Calculate total value per item ordered

```
Input: select order_item_id , (quantity * item_price) as total_value_per_item from
order_items;
```

Output:

order_item_id	total_value_per_item
1	9414.28
2	532.20
3	2955.57
4	2208.99
5	9469.78
6	4707.14
7	9794.72
8	14692.08
9	1429.45
10	723.97
11	14204.67
12	3043.67
13	14405.34
14	14167.98
15	4140.74
16	14692.08
17	396.11
18	9131.01
19	4288.35
20	639.43

Result 14

--4. Combine customer name and phone number

```
Input: select concat( name, ' - ', phone) as customer_contact from customers;
```

Output:

customer_contact
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7
Walter Jenkins - 536-329-0817x71
Mary Knight - 361-636-3802
Leslie Wilson - +1-256-261-1984
Deborah Arias - 811-821-2144x97
Austin Flores - 329.901.1576x66
Amy Landry - +1-278-019-3748
Randy Mooney - (158)927-7313x3
Jeffrey Bray - 164.962.0222x92
Amanda Bright - 380.981.9798x69
Megan Lee - 7044478333

Result 15

--5. Extract only the date part from order timestamps

```
Input: select order_id, customer_id, date(order_date) as 'date',status,total_amount
from orders;
```

Output:

order_id	customer_id	date	status	total_amount
1	20	2025-03-02	Delivered	9414.28
2	18	2024-10-09	Shipped	532.20
3	15	2025-05-08	Cancelled	5164.56
4	11	2024-09-19	Delivered	9469.78
5	12	2025-04-08	Pending	14501.86
6	29	2024-10-25	Cancelled	31050.17
7	22	2024-07-29	Shipped	3043.67
8	19	2024-07-30	Cancelled	32714.06
9	6	2025-06-10	Pending	24219.20
10	28	2025-02-16	Delivered	24342.52
11	25	2025-03-11	Cancelled	16196.13
12	28	2025-01-15	Shipped	9890.72
13	25	2025-05-12	Cancelled	9627.36
14	1	2024-09-24	Shipped	15803.34
15	29	2024-07-18	Pending	12421.88
16	5	2024-09-09	Shipped	22856.73
17	1	2024-08-19	Pending	11173.77
18	13	2024-06-15	Cancelled	32001.24
19	29	2024-07-06	Cancelled	843.46
20	20	2024-08-24	Delivered	6400.68

Result 16

--6. List products that do not have any stock left

```
Input: select * from products where stock_quantity =0;
```

Output:

product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

-- Level 3: Aggregations

-- 1. Count the total number of orders placed

```
Input: select count(order_id) from orders;
```

Output:

count(order_id)
400

-- 2. Calculate the total revenue collected from all orders

```
Input: select sum(amount_paid) as total_revenue from payments;
```

Output:

total_revenue
6960973.66

-- 3. Calculate the average order value

```
Input: select round(avg(total_amount),2) as average_order from orders;
```

Output:

average_order
17402.43

-- 4. Count the number of customers who have placed at least one order

```
Input: select count(distinct(customer_id)) as active_customer from orders;
```

Output:

active_customer
30

-- 5. Find the number of orders placed by each customer

```
Input: select customer_id , count(order_id) AS TOTAL_ORDER from orders group by
customer_id;
```

Output:

customer_id	TOTAL_ORDER
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11
13	12
14	15

Result 23 ×

-- 6. Find total sales amount made by each customer

```
Input: select customer_id, sum(total_amount) as TOTAL_AMOUNT_EACH_CUSTOMER from
orders group by customer_id;
```

Output:

customer_id	TOTAL_AMOUNT_EACH_CUSTOMER
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79
13	152727.70
14	360324.18

Result 24

-- 7. List the number of products sold per category

```
Input:select category, count(product_id) as product_count from products group by
category;
```

Output:

category	product_count
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

-- 8. Find the average item price per category

```
Input:select category, round(avg(price),2) as AVG_ITEM_PRICE from products group by
category;
```

Output:

category	AVG_ITEM_PRICE
Home	2146.37
Clothing	3434.58
Electronics	2653.39
Toys	2516.53
Books	3167.08

-- 9. Show number of orders placed per day

```
Input: select date(order_date), count(order_id) as ORDER_PER_DAY from orders group
by date(order_date) order by ORDER_PER_DAY desc;
```

Output:

	date(order_date)	ORDER_PER_DAY
▶	2024-12-07	5
	2025-06-06	5
	2025-01-15	4
	2025-05-05	4
	2024-11-10	4
	2025-02-07	4
	2024-09-23	4
	2025-03-02	3
	2024-09-19	3
	2025-04-08	3
	2024-10-25	3
	2025-03-11	3
	2024-09-09	3
	2024-08-24	3

Result 30 ×

-- 10. List total payments received per payment method

```
Input: select method, sum(amount_paid) as payment_recieved from payments group by
method order by payment_recieved desc;
```

Output:

	method	payment_recieved
▶	Debit Card	1930577.88
	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78

-- Level 4: JOINS

-- 1. Retrieve order details along with the customer name (INNER JOIN)

```
Input: select o.order_id, c.name, o.total_amount from customers c
inner join orders o on o.customer_id = c.customer_id;
```

Output:

order_id	name	total_amount
14	Thomas Owens	15803.34
17	Thomas Owens	11173.77
61	Thomas Owens	13053.00
76	Thomas Owens	23506.81
92	Thomas Owens	834.75
109	Thomas Owens	12190.14
127	Thomas Owens	20160.64
135	Thomas Owens	8891.79
144	Thomas Owens	12093.54
221	Thomas Owens	12437.61
278	Thomas Owens	32015.16
379	Thomas Owens	21586.89
56	Charles Grant	6828.61
169	Charles Grant	9426.30

Result 32

-- 2. Get list of products that have been sold (INNER JOIN with order_items)

```
Input: select distinct p.product_id, p.name, p.category from products p
inner join order_items o on p.product_id = o.product_id;
```

Output:

product_id	name	category
1	Plant No	Home
2	Population Social	Clothing
3	Available Answer	Electronics
4	Any Question	Clothing
5	Natural Network	Toys
6	If Whatever	Electronics
7	Response Indeed	Clothing
8	Every Amount	Home
9	Common Study	Toys
10	Development System	Electronics
11	Build Her	Books
12	Action Ask	Electronics
13	Full West	Books
14	Listen Development	Home

Result 33

-- 3. List all orders with their payment method (INNER JOIN)

```
Input:select o.*, p.method from orders o inner join payments p on o.order_id =
p.order_id;
```

Output:

order_id	customer_id	order_date	status	total_amount	method
1	20	2025-03-02 07:20:11	Delivered	9414.28	Credit Card
2	18	2024-10-09 18:08:21	Shipped	532.20	Net Banking
3	15	2025-05-08 00:08:27	Cancelled	5164.56	Credit Card
4	11	2024-09-19 22:16:13	Delivered	9469.78	UPI
5	12	2025-04-08 18:02:06	Pending	14501.86	UPI
6	29	2024-10-25 07:33:59	Cancelled	31050.17	UPI
7	22	2024-07-29 11:58:47	Shipped	3043.67	UPI
8	19	2024-07-30 22:49:49	Cancelled	32714.06	Net Banking
9	6	2025-06-10 17:00:25	Pending	24219.20	Net Banking
10	28	2025-02-16 12:45:59	Delivered	24342.52	Debit Card
11	25	2025-03-11 14:22:46	Cancelled	16196.13	Credit Card
12	28	2025-01-15 13:22:53	Shipped	9890.72	Credit Card
13	25	2025-05-12 13:04:29	Cancelled	9627.36	UPI
14	1	2024-09-24 21:21:38	Shipped	15803.34	UPI

Result 34

-- 4. Get list of customers and their orders (LEFT JOIN)

```
Input: select c.name, o.order_id from customers c
left join orders o on c.customer_id= o.customer_id;
```

Output:

name	order_id
Thomas Owens	14
Thomas Owens	17
Thomas Owens	61
Thomas Owens	76
Thomas Owens	92
Thomas Owens	109
Thomas Owens	127
Thomas Owens	135
Thomas Owens	144
Thomas Owens	221
Thomas Owens	278
Thomas Owens	379
Charles Grant	56
Charles Grant	169

Result 35

-- 5. List all products along with order item quantity (LEFT JOIN)

```
Input: select p.name as product_name, p.category, sum(o.quantity) as total_quantity
from products p left join order_items o on p.product_id = o.product_id group by
p.category , p.name;
```

Output:

product_name	category	total_quantity
Plant No	Home	43
Population Social	Clothing	33
Available Answer	Electronics	47
Any Question	Clothing	30
Natural Network	Toys	44
If Whatever	Electronics	54
Response Indeed	Clothing	39
Every Amount	Home	61
Common Study	Toys	57
Development System	Electronics	53
Build Her	Books	50
Action Ask	Electronics	47
Full West	Books	46
Listen Development	Home	43

Result 36

-- 6. List all payments including those with no matching orders (RIGHT JOIN)

```
Input: select p.* from orders o right join payments p on o.order_id = p.order_id
where o.order_id is null;
```

Output:

payment_id	order_id	payment_date	amount_paid	method

-- 7. Combine data from three tables: customer, order, and payment

```
Input: select c.name, o.order_id , o.total_amount, p.method, date(payment_date) as
order_date from customers c join orders o on c.customer_id = o.customer_id join
payments p on o.order_id = p.order_id;
```

Output:

name	order_id	total_amount	method	order_date
Thomas Owens	14	15803.34	UPI	2024-09-24
Thomas Owens	17	11173.77	UPI	2024-08-19
Thomas Owens	61	13053.00	Net Banking	2024-12-26
Thomas Owens	76	23506.81	Credit Card	2025-01-14
Thomas Owens	92	834.75	Net Banking	2024-09-26
Thomas Owens	109	12190.14	UPI	2025-03-17
Thomas Owens	127	20160.64	Debit Card	2025-06-10
Thomas Owens	135	8891.79	Debit Card	2025-03-11
Thomas Owens	144	12093.54	UPI	2024-09-19
Thomas Owens	221	12437.61	UPI	2024-09-25
Thomas Owens	278	32015.16	UPI	2025-02-01
Thomas Owens	379	21586.89	Credit Card	2025-03-22
Charles Grant	56	6828.61	Credit Card	2024-06-23
Charles Grant	169	9426.30	Credit Card	2025-02-11

Result 38

-- Level 5: Subqueries

-- 1. List all products priced above the average product price

```
Input: select * from products
where price > (select avg(price) from products) order by price desc;
```

Output:

product_id	name	category	price	stock_quantity	added_on
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

products 39 ×

-- 2. Find customers who have placed at least one order

```
Input: select * from customers c where (select count(*) from orders o where
o.customer_id = c.customer_id) > 0;
```

Output:

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28

customers 40

-- 3. Show orders whose total amount is above the average for that customer

```
Input: select customer_id, order_id, sum(total_amount) as total_spent from orders
group by customer_id, order_id having total_spent > (select avg(amount_paid) from
payments) order by total_spent desc;
```

Output:

customer_id	order_id	total_spent
14	107	48042.06
28	393	44504.56
29	309	43867.08
2	315	42056.04
29	225	41885.52
3	281	41679.11
29	300	41322.58
2	223	41020.75
17	256	40944.10
25	189	40872.29
15	243	40510.54
5	266	39921.78
18	139	39857.19
20	327	39563.51

Result 42

-- 4. Display customers who haven't placed any orders

Input: select c.* from customers c left join orders o on c.customer_id = o.customer_id where o.order_id is null;

Output:

customer_id	name	email	phone	created_at

-- 5. Show products that were never ordered

```
Input: select p.* from products p left join order_items oi on p.product_id =
oi.product_id where oi.product_id is null;
```

Output:

product_id	name	category	price	stock_quantity	added_on

-- 6. Show highest value order per customer

```
Input: select o.customer_id, o.order_id, o.total_amount from orders o where
o.total_amount = (select max(o2.total_amount) from orders o2 where o2.customer_id =
o.customer_id ) order by o.total_amount desc;
```

Output:

customer_id	order_id	total_amount
14	107	48042.06
28	393	44504.56
29	309	43867.08
2	315	42056.04
3	281	41679.11
17	256	40944.10
25	189	40872.29
15	243	40510.54
5	266	39921.78
18	139	39857.19
20	327	39563.51
6	330	39003.19
26	258	38836.44
12	80	38656.26

orders 46 ×

-- 7. Highest Order Per Customer (Including Names)

```
Input: select c.name, o.customer_id, o.order_id, o.total_amount as highest_order
from orders o join customers c on o.customer_id = c.customer_id where o.total_amount
= (select max(o2.total_amount) from orders o2 where o2.customer_id = o.customer_id)
order by highest_order desc;
```

Output:

name	customer_id	order_id	highest_order
Deborah Arias	14	107	48042.06
Joseph Stuart	28	393	44504.56
Kara Zavala	29	309	43867.08
Charles Grant	2	315	42056.04
Kaitlin Richards	3	281	41679.11
Randy Mooney	17	256	40944.10
Cindy Hart	25	189	40872.29
Austin Flores	15	243	40510.54
David Allen	5	266	39921.78
Jeffrey Bray	18	139	39857.19
Megan Lee	20	327	39563.51
Mark Duke	6	330	39003.19
Victoria Hall	26	258	38836.44
Mary Knight	12	80	38656.26

Result 48

-- Level 6: Set Operations

-- 1. List all customers who have either placed an order or written a product review

```
Input: select distinct c.customer_id, c.name, c.email from customers c left join
orders o on c.customer_id = o.customer_id left join product_reviews pr on
c.customer_id = pr.customer_id where o.customer_id is not null or pr.customer_id is
not null;
```

Output:

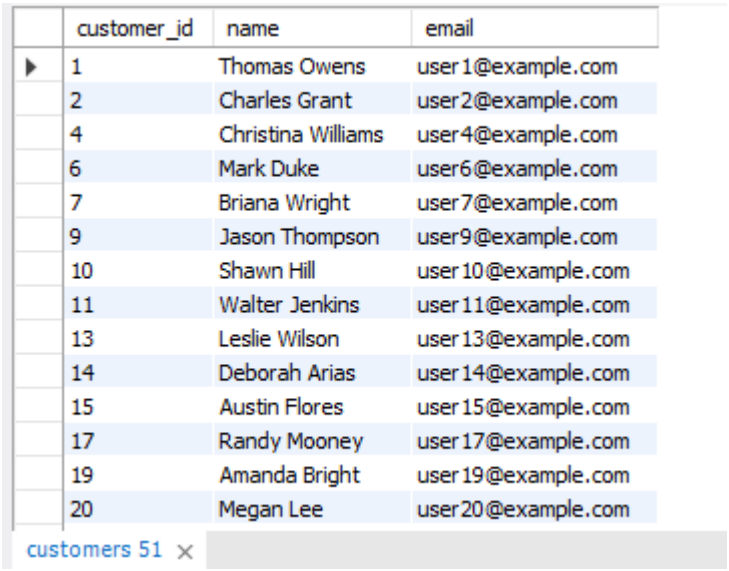
customer_id	name	email
1	Thomas Owens	user1@example.com
2	Charles Grant	user2@example.com
3	Kaitlin Richards	user3@example.com
4	Christina Williams	user4@example.com
5	David Allen	user5@example.com
6	Mark Duke	user6@example.com
7	Briana Wright	user7@example.com
8	John Bryan	user8@example.com
9	Jason Thompson	user9@example.com
10	Shawn Hill	user10@example.com
11	Walter Jenkins	user11@example.com
12	Mary Knight	user12@example.com
13	Leslie Wilson	user13@example.com
14	Deborah Arias	user14@example.com

Result 49

-- 2. List all customers who have placed an order as well as reviewed a product [intersect not supported]

```
Input: select customer_id, name, email from customers
where customer_id in (select customer_id from orders)
and customer_id in (select customer_id from product_reviews);
```

Output:



customer_id	name	email
1	Thomas Owens	user1@example.com
2	Charles Grant	user2@example.com
4	Christina Williams	user4@example.com
6	Mark Duke	user6@example.com
7	Briana Wright	user7@example.com
9	Jason Thompson	user9@example.com
10	Shawn Hill	user10@example.com
11	Walter Jenkins	user11@example.com
13	Leslie Wilson	user13@example.com
14	Deborah Arias	user14@example.com
15	Austin Flores	user15@example.com
17	Randy Mooney	user17@example.com
19	Amanda Bright	user19@example.com
20	Megan Lee	user20@example.com

customers 51 ×

Conclusion

This SQL project demonstrates the practical implementation of database concepts such as filtering, joins, aggregations, subqueries, and reporting using a Retail Shop database. The project helps in understanding how SQL can be used to solve real-world business problems.